

BoilerTronics Technical Manual: Reduced Instruction Set Language

Specification, Syntax, and Semantics Overview

BoilerTronics' Engineer Trapped in a Basement

November 5, 2025

Contents

1	Overview	3
1.1	Execution	3
2	Registers	3
3	Instruction Classes	4
3.1	Step-Consuming Instructions	4
3.2	Free Instructions	4
4	Instruction Syntax and Semantics	4
4.1	Arithmetic Instructions	4
4.2	Write Instructions	5
4.3	Jump Instructions	5
4.4	Wait Instruction	5
4.5	Movement Instructions	6
5	Program Structure and Labels	6
6	Execution Semantics	6

7	Error Handling	7
8	Example Program	7
9	Conclusion	7

1 Overview

This document describes the reduced instruction set assembly language used for all programmable elements. The language is designed to control and automate mechanical entities, including `ClawObject`, `ConveyorRotatorObject`, and other `Scriptable` objects on the BoilerTronics factory floor.

The language supports a set of arithmetic, control, and motion instructions that can be executed line-by-line. The system uses register-based computation with a small number of general-purpose registers and a single comparison register. Arithmetic and comparative operations do not consume a time step, movement commands do.

1.1 Execution

The programs are executed linearly, starting from the program counter (PC) = 0, unless modified by a control-flow instruction. Execution proceeds until:

- a **halt condition** is reached (end of program),
- a **step-consuming instruction** is executed (causing physical time progression),
- or an **execution error** occurs,
- Manual **interruption** in which the user pauses the level or resets the level state.

2 Registers

The language defines four internal integer registers:

Register	Purpose
r0	Primary accumulator register (arithmetic results).
r1	General-purpose register.
r2	General-purpose register.
cmp	Comparison result register: -1 (less), 0 (equal), 1 (greater).

All registers initialize to zero at the start or reset of the program. Registers can store integer values and be referenced directly by their names in instructions.

Since the BoilerTronics Factory cannot afford quality machines, memory is inherently unstable. Upon a write, the value in a register will be preserved for 5 time steps before becoming corrupted into a null value. Rewriting to the register will remove the null value.

3 Instruction Classes

The instructions are categorized as *free* (zero-time) or *step-consuming*. Free instructions execute immediately within a logical cycle. Step-consuming instructions represent physical operations and pause program progression after execution.

3.1 Step-Consuming Instructions

- `mov [l|r|u|d]`
- `rot [l|r]`
- `grb`
- `drp`
- `wait`

3.2 Free Instructions

- `add, sub, mul, div, cmp`
- `jmp, jeq, jne, jgt, jlt, jge, jle`
- `wrt`

4 Instruction Syntax and Semantics

This section details the accepted syntax (in regular-expression form) and semantics for each instruction.

4.1 Arithmetic Instructions

```
add <op1> <op2>
sub <op1> <op2>
mul <op1> <op2>
div <op1> <op2>
cmp <op1> <op2>
```

Operands may be literal integers or register names (`r0`, `r1`, `r2`, `cmp`). The result of arithmetic operations is stored in `r0`. Division by zero raises a runtime error. Operations besides a write involving a null register will result in a null exception. For `cmp`, the result is written to `cmp` according to:

$$cmp = \begin{cases} 0 & \text{if } op1 = op2 \\ -1 & \text{if } op1 < op2 \\ 1 & \text{if } op1 > op2 \end{cases}$$

Example:

```
wrt r1 3
wrt r2 5
add r1 r2 ; r0 = 8
cmp r1 r2 ; cmp = -1
```

4.2 Write Instructions

Two forms of `wrt` exist:

```
wrt <reg> <int>
wrt <reg> <reg>
```

This writes a literal or another register's value into the destination register.

Example:

```
wrt r1 10
wrt r2 r1 ; r2 = 10
```

4.3 Jump Instructions

Control flow operations are label-based. Labels are mapped to line indices during pre-processing.

```
jmp <label>
jeq <label> ; jump if cmp == 0
jne <label> ; jump if cmp != 0
jgt <label> ; jump if cmp > 0
jlt <label> ; jump if cmp < 0
jge <label> ; jump if cmp >= 0
jle <label> ; jump if cmp <= 0
```

Labels are defined using the preprocessing stage and resolved into an internal line index map. Undefined labels trigger an execution error.

Example:

```
start:
wrt r1 0
loop:
add r1 1
cmp r1 10
jlt loop
```

4.4 Wait Instruction

```
wait
```

Suspends program execution for one time step. Used for timing synchronization or staged movement. Or you could waste time steps.

4.5 Movement Instructions

These commands directly control object behaviors, consuming time steps.

Move

```
mov l | mov r | mov u | mov d
```

Moves the associated object left, right, up, or down.

Rotate

```
rot l | rot r
```

Rotates the object left or right. Only valid for `ConveyorRotatorObject`.

Grab and Drop

```
grb  
drp
```

Manipulates held items. Only valid for `ClawObject`. Invalid use raises a runtime error.

5 Program Structure and Labels

A valid program consists of lines matching the above instruction syntaxes. Empty or comment lines are ignored. Labels are declared inline using the format:

```
label_name:
```

Each label must be unique. During preprocessing, the interpreter maps label names to their corresponding instruction indices.

6 Execution Semantics

The interpreter maintains a **program counter** (PC) that tracks the current instruction index. The following defines the operational semantics for each instruction class:

- **Free instructions:** execute atomically and increment PC by one.
- **Jump instructions:** modify PC to the target label index if the condition is met.
- **Step-consuming instructions:** execute the corresponding physical action and halt progression for that tick.

If the PC exceeds the number of valid lines, the program halts.

7 Error Handling

The interpreter performs runtime error detection for the following conditions:

- Invalid instruction syntax or undefined opcode.
- Invalid register or label reference.
- Division by zero.
- Invalid command usage for a given object type.

Upon error, an error event is raised to the system's event manager and execution halts.

8 Example Program

Below is a complete example demonstrating arithmetic, control, and motion integration:

```
start:
    wrt r1 0
    wrt r2 3
loop:
    add r1 r2 ; accumulate
    cmp r1 20
    jlt loop ; repeat until >= 20
    wait ; pause for one step
    grb ; grab object
    mov r ; move right
    drp ; drop object
```

This program accumulates register values, waits, and then performs a mechanical sequence.

9 Conclusion

If you are still reading this, abandon all hope. Or don't. Your call.